

ALL GATES

Inputs

Input	Width	MSB	LSB
a	1 bit	a	a
b	1 bit	b	b

Since a and b are only 1 bit, they do not have separate MSB and LSB. The same bit is both MSB and LSB.

Outputs

Output	Gate	Width	MSB	LSB
y1	AND	1 bit	y1	y1
y2	OR	1 bit	y2	y2
y3	NAND	1 bit	y3	y3
y4	NOR	1 bit	y4	y4
y5	XOR	1 bit	y5	y5
y6	XNOR	1 bit	y6	y6

MAIN CODE

```
module allgates(  
    input wire a,  
    input wire b,  
    output wire y_and,  
    output wire y_or,  
    output wire y_nand,  
    output wire y_nor,  
    output wire y_xor,  
    output wire y_xnor,  
    output wire y_not  
);  
  
assign y_and = a & b;  
assign y_or = a | b;
```

```
assign y_nand = ~(a & b);  
assign y_nor = ~(a | b);  
assign y_xor = a ^ b;  
assign y_xnor = ~(a ^ b);  
assign y_not = ~a;
```

```
endmodule
```

TEST BENCH CODE

```
module allgates_tb;  
  
    reg a;  
    reg b;  
  
    wire y_and;  
    wire y_or;  
    wire y_nand;  
    wire y_nor;  
    wire y_xor;  
    wire y_xnor;  
    wire y_not;  
  
    // Instantiate the design  
    allgates uut (  
        .a(a),  
        .b(b),  
        .y_and(y_and),  
        .y_or(y_or),  
        .y_nand(y_nand),
```

```
.y_nor(y_nor),  
.y_xor(y_xor),  
.y_xnor(y_xnor),  
.y_not(y_not)  
);
```

```
initial begin
```

```
// Test 1: a=0, b=0
```

```
a = 1'b0;
```

```
b = 1'b0;
```

```
#10;
```

```
// Test 2: a=0, b=1
```

```
a = 1'b0;
```

```
b = 1'b1;
```

```
#10;
```

```
// Test 3: a=1, b=0
```

```
a = 1'b1;
```

```
b = 1'b0;
```

```
#10;
```

```
// Test 4: a=1, b=1
```

```
a = 1'b1;
```

```
b = 1'b1;
```

```
#10;
```

```
$finish;
```

```
end
```

endmodule